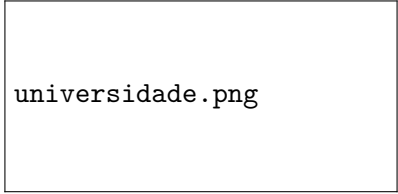


Universidade do Minho

Licenciatura em Engenharia Informática



universidade.png

Processamento de Linguagens

2025/2026

Relatório Técnico

Grupo 10

A106877, José Miguel Fernandes Cação

A106793, Lucas André Dias Fernandes

A106918, Rafael Filipe Duarte de Andrade

Maio 2026

Índice

1	Introdução	2
2	Arquitetura do Compilador	2
3	Pré-processamento	4
4	Análise Léxica	4
4.1	Tokens reconhecidos	4
5	Análise Sintática e Gramática	4
5.1	Gramática (resumo)	5
5.2	Precedência de operadores	5
5.3	AST	5
6	Representação Intermédia e Otimização (TAC)	6
6.1	Lowering	6
6.2	Otimizações aplicadas	6
7	Análise Semântica	6
8	Geração de Código VM	7
9	Geração de Código VM	7
9.1	Modelo de memória	8
9.2	Inlining de subprogramas	8
9.3	Exemplo de saída	8
10	Testes	9
11	Dificuldades Encontradas	9
12	Como Correr o Compilador	9
12.1	Dependências	9
12.2	Compilar um programa Fortran	10
12.3	Correr os testes	10
13	Conclusão	10

1 Introdução

O presente relatório descreve o compilador para a linguagem **Fortran 77** (standard ANSI X3.9-1978) desenvolvido no âmbito do projeto de Processamento de Linguagens 2026.

O compilador traduz um subconjunto representativo de Fortran 77 para código da máquina virtual disponibilizada na unidade curricular, percorrendo as etapas clássicas de um compilador moderno: pré-processamento, análise léxica, análise sintática, análise semântica, otimização por código de três endereços (*TAC*) e geração de código.

Funcionalidades suportadas

- Tipos de dados escalares: `INTEGER`, `REAL`, `LOGICAL`.
- Arrays unidimensionais de qualquer um dos tipos anteriores.
- Expressões aritméticas, relacionais e lógicas com precedência correcta.
- Comandos de controlo de fluxo: `IF / THEN / ELSE / ENDIF`, `DO` com label, `GOTO` e labels numéricas.
- Entrada/saída básica: `READ *`, e `PRINT *,.`
- Subprogramas externos: `FUNCTION` tipada e `SUBROUTINE`.
- Função embutida `MOD(a, b)`.
- Pré-processamento automático de *fixed-form* e *free-form*.

2 Arquitetura do Compilador

O compilador está organizado em módulos independentes, ligados por um *pipeline* linear implementado em `src/compiler.py`:

1. **Pré-processamento** - detecção automática do formato e normalização para *free-form*.
2. **Análise léxica** - tokenização com `ply.lex`.
3. **Análise sintática** - construção da AST com `ply.yacc`.
4. **Otimização TAC** - simplificação de expressões por *three-address code*.
5. **Análise semântica** - validação de tipos, declarações e consistência de controlo de fluxo.
6. **Geração de código** - emissão de instruções da VM a partir da AST validada.

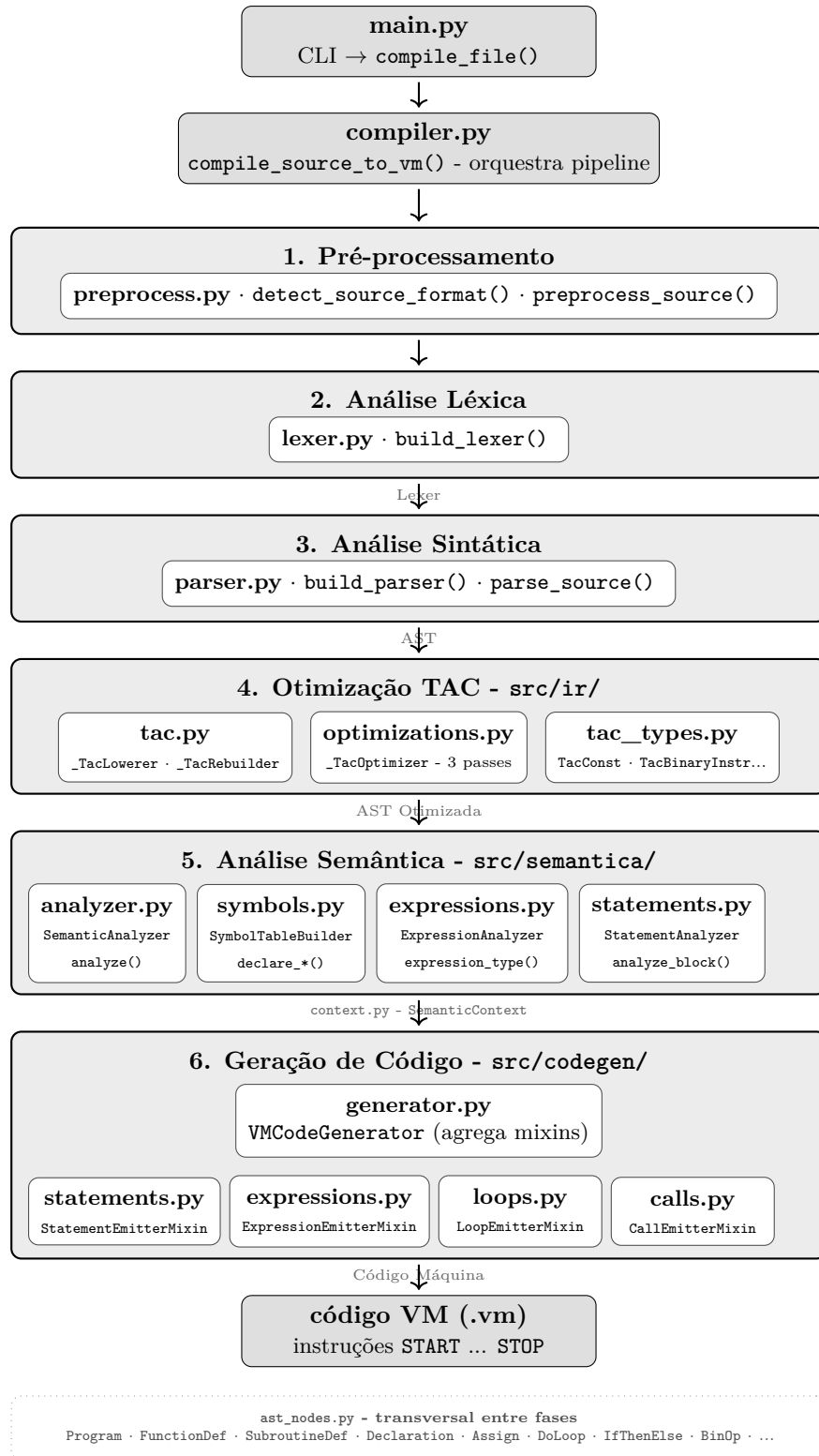


Figura 1: Pipeline detalhado e mapeamento de módulos do compilador.

A separação em módulos facilita a manutenção e os testes: cada fase pode ser testada de forma isolada sem conhecer as restantes.

3 Pré-processamento

O Fortran 77 *standard* usa o formato de **colunas fixas**:

- Colunas 1–5: label opcional (apenas dígitos ou espaços).
- Coluna 6: marcador de continuação (qualquer caráter exceto espaço ou 0).
- Colunas 7–72: código Fortran.
- C, c, * ou ! na coluna 1: comentário.

O módulo `src/preprocess.py` **deteta automaticamente** o formato através de um sistema de pontuação heurística: linhas com campos de label numérica válidos e coluna 6 preenchida pontuam a favor de *fixed-form*; o marcador & no fim de uma linha é sinal decisivo de *free-form*. Em caso de empate, prevalece o *free-form*.

Após a detecção, o pré-processador:

- Em *fixed-form*: extrai o label, ignora a coluna de continuação e une linhas lógicas.
- Em *free-form*: remove comentários ! (respeitando strings com apostrofes e o escape ") e resolve continuações com &.

O resultado é sempre uma sequência de linhas lógicas que o lexer consegue processar diretamente.

4 Análise Léxica

O lexer é implementado com `ply.lex` em `src/lexica/lexer.py`.

4.1 Tokens reconhecidos

Categoria	Tokens
Palavras reservadas	PROGRAM, INTEGER, REAL, LOGICAL, IF, THEN, ELSE, ENDIF, DO, GOTO, CONTINUE, READ, PRINT, END, FUNCTION, SUBROUTINE, CALL, RETURN
Identificadores	ID
Literais numéricos	NUMBER (inteiro), FNUMBER (real)
Literais lógicos	TRUE (.TRUE.), FALSE (.FALSE.)
Strings	STRING (delimitada por apostrofes)
Operadores aritméticos	PLUS, MINUS, TIMES, DIVIDE
Operadores relacionais	EQ_OP, NE_OP, LT_OP, LE_OP, GT_OP, GE_OP
Operadores lógicos	AND_OP, OR_OP, NOT_OP
Pontuação	EQUALS, LPAREN, RPAREN, COMMA
Estrutural	NEWLINE

Tabela 1: Tokens do subconjunto de Fortran 77 suportado.

Todos os identificadores e palavras-chave são normalizados para maiúsculas, pelo que `program`, `PROGRAM` e `Program` são equivalentes.

5 Análise Sintática e Gramática

O parser é implementado com `ply.yacc` em `src/sintatica/parser.py`. Cada regra gramatical corresponde a uma função `p_...`, cujo docstring é lido pelo PLY como a produção EBNF.

5.1 Gramática (resumo)

Listing 1: Gramática simplificada do subconjunto.

```

1 program      ::= PROGRAM ID NEWLINE lines END external_defs_opt
2
3 external_def ::= function_def | subroutine_def
4 function_def ::= type_spec FUNCTION ID ( param_list_opt ) NEWLINE lines END
5 subroutine_def ::= SUBROUTINE ID ( param_list_opt ) NEWLINE lines END
6
7 declaration  ::= type_spec decl_list
8 type_spec    ::= INTEGER | REAL | LOGICAL
9 decl_item    ::= ID | ID ( NUMBER )
10
11 statement    ::= ID = expression
12                | ID ( expression ) = expression
13                | PRINT *, print_list
14                | READ *, read_target_list
15                | GOTO NUMBER
16                | CONTINUE
17                | RETURN
18                | CALL ID ( call_args_opt )
19                | DO NUMBER ID = expression , expression [, expression]
20                | if_statement
21
22 if_statement ::= IF ( expression ) THEN NEWLINE block_lines
23                [ELSE NEWLINE block_lines]
24                ENDIF NEWLINE
25
26 expression   ::= NUMBER | FNUMBER | .TRUE. | .FALSE. | ID
27                | ID ( expression )
28                | ID ( call_args )
29                | expression op expression
30                | - expression
31                | .NOT. expression
32                | ( expression )
33
34 op           ::= + | - | * | / | .EQ. | .NE. | .LT. | .LE.
35                | .GT. | .GE. | .AND. | .OR.

```

5.2 Precedência de operadores

A tabela de precedência do PLY (do menor para o maior) é:

Nível	Operadores
1 (menor)	.OR.
2	.AND.
3	.EQ. .NE. .LT. .LE. .GT. .GE.
4	+ -
5	* /
6 (maior)	.NOT. menos unário

5.3 AST

O resultado do parser é uma *Abstract Syntax Tree* composta por *dataclasses* imutáveis definidas em `src/ast_nodes.py`: `Program`, `FunctionDef`, `SubroutineDef`, `Declaration`, `IfThenElse`,

DoLoop, BinOp, entre outros. A imutabilidade (`frozen=True`) garante que nenhuma fase modifica a AST produzida por outra fase.

6 Representação Intermédia e Otimização (TAC)

Entre o parser e o gerador de código existe uma etapa de otimização baseada em **TAC** (*three-address code*), implementada em `src/ir/`.

6.1 Lowering

A classe `_TacLowerer` converte cada expressão AST em instruções TAC com temporários (`t1`, `t2`, ...):

Listing 2: Expressão e TAC correspondente.

```

1 A + B * 2
2 => t1 = B * 2
3     t2 = A + t1

```

6.2 Otimizações aplicadas

O `_TacOptimizer` aplica três otimizações locais:

- **Constant folding:** operações entre constantes são avaliadas em tempo de compilação (*e.g.*, $2 * 3 \rightarrow 6$, $10 / 2 \rightarrow 5$). A divisão inteira trunca para zero, tal como no Fortran standard. Divisão por zero não é dobrada e mantém-se como instrução para a VM.
- **Propagação de cópias:** um temporário que recebe directamente o valor de outro é eliminado.
- **Eliminação de código morto:** temporários cujo valor nunca é lido são removidos das instruções.

Após a otimização, a classe `_TacRebuilder` reconstrói uma AST simplificada que é entregue ao gerador de código.

7 Análise Semântica

A análise semântica (módulo `src/semantica/`) percorre a AST e verifica:

- **Declaração de variáveis:** toda a variável usada tem de estar declarada; redeclarações são erro.
- **Consistência de labels:** labels duplicadas são rejeitadas; GOTO para label não definida é erro; a label de fecho de cada DO tem de corresponder a um CONTINUE existente.
- **Verificação de tipos:** operandos de operações aritméticas devem ser INTEGER ou REAL; operandos de .AND., .OR., .NOT. devem ser LOGICAL; índices de arrays têm de ser INTEGER.
- **Subprogramas:** aridade e tipos dos argumentos na chamada devem coincidir com a declaração da função/subrotina.

A fase semântica devolve três tabelas ao gerador de código: os símbolos globais (nome $\rightarrow$ tipo, índice na memória, dimensão), os metadados das funções e os das subrotinas.

8 Geração de Código VM

O gerador de código (`src/codegen/`) é a etapa final do pipeline e tem como responsabilidade emitir as instruções lineares em formato de texto aceites pela máquina virtual baseada em pilha.

Uma decisão de design central na nossa arquitetura foi fazer com que o gerador opere diretamente sobre a *AST já otimizada pelo TAC*, enriquecida pelas tabelas de símbolos e metadados validados na fase de Análise Semântica. Esta abordagem garante que o gerador não necessita de recalcular escopos ou inferir tipos primitivos; a AST dita a estrutura hierárquica e o fluxo de controlo do programa, enquanto os dicionários semânticos fornecem os mapeamentos exatos de endereçamento na VM (`base_index`) e os blocos de código isolados necessários para a expansão em linha (*inlining*).

Assim, esta abordagem foi uma escolha estratégica fundamentada nos seguintes fatores técnicos:

1. **Preservação do Controlo de Fluxo Estruturado:** O nosso módulo TAC foi desenhado especificamente para otimizações locais e simplificações algébricas no seio de expressões aritméticas e lógicas. Ele não efetua a linearização do fluxo de controlo (como a transformação de ciclos em grafos de fluxo de controlo com *Basic Blocks*). Por conseguinte, a AST é o único local onde a hierarquia estrutural de alto nível do Fortran 77 (como os blocos `DoLoop` e `IfThenElse`) permanece intacta e explicitamente representada.
2. **Tradução Dirigida pela Sintaxe (Syntax-Directed Translation):** Ao mantermos a AST como a estrutura de dados de entrada da geração de código, conseguimos implementar um gerador modular e elegante baseado no padrão de *Syntax-Directed Translation*. O gerador percorre recursivamente a árvore através de *mixins* especializados que sabem exatamente como traduzir cada nodo abstrato em *opcodes* equivalentes da VM, mantendo o compilador altamente robusto e legível.
3. **Facilidade de Inlining:** A técnica de inlining adotada para a expansão de subprogramas (`FUNCTION` e `SUBROUTINE`) torna-se trivial ao nível da AST, pois permite duplicar e injetar sequências inteiras de nós estruturados (*statements*) no ponto exato da chamada, uma tarefa que seria exponencialmente mais complexa e propensa a erros se fosse efetuada sobre um código intermédio linear plano.

Para manter o código limpo, coeso e evitar um ficheiro monolítico, a classe principal `VMCodeGenerator` foi estruturada através da composição de vários *mixins* especializados nas subregras da linguagem:

- `StatementEmitterMixin` - tradução de atribuições, `PRINT`, `READ`, `GOTO` e mapeamento de labels.
- `ExpressionEmitterMixin` - tradução de literais, variáveis e avaliação de expressões binárias/unárias na pilha.
- `LoopEmitterMixin` - gestão e emissão de ciclos `DO` com rótulos internos (`DOCHECK` e `DOEXIT`), seleccionando *opcodes* inteiros (`INFEQ`) ou reais (`FINFEQ`) conforme o tipo da variável de controlo.
- `CallEmitterMixin` - orquestração do inlining e resolução da função embutida `MOD`.

9 Geração de Código VM

O gerador de código (`src/codegen/`) emite instruções para a máquina virtual baseada em pilha. A classe `VMCodeGenerator` é composta por *mixins* especializados:

- `StatementEmitterMixin` - atribuições, `PRINT`, `READ`, `GOTO`, labels.
- `ExpressionEmitterMixin` - literais, variáveis, operações binárias e unárias.
- `LoopEmitterMixin` - ciclos `DO` com a geração de rótulos internos `DOCHECK` e `DOEXIT`; selec-

ciona automaticamente opcodes inteiros (INFEQ, ADD) ou reais (FINFEQ, FADD) conforme o tipo da variável de controlo.

- CallEmitterMixin - inlining de FUNCTION e SUBROUTINE.

9.1 Modelo de memória

Todas as variáveis do programa principal são armazenadas em slots globais consecutivos (PUSHG *n*/STOREG *n*), reservados pelo prólogo PUSHNN. Arrays ocupam *N* slots contíguos, indexados com PUSHG *base* + *índice*.

9.2 Inlining de subprogramas

A estratégia adotada para FUNCTION e SUBROUTINE é o **inlining**: o corpo do subprograma é emitido em linha no ponto de chamada, com as variáveis locais em slots globais dedicados. Esta opção simplifica significativamente o gerador (não requer pilha de chamadas ou registos de activação) e é suficiente para o subconjunto sem recursão.

9.3 Exemplo de saída

O programa *Fatorial* (examples/fatorial_do.f77) produz:

Listing 3: Saída VM do programa Fatorial.

```
1 START
2 PUSHN 3
3 PUSHS "Introduza um numero inteiro positivo:"
4 WRITES
5 WRITELN
6 READ
7 ATOI
8 STOREG 0
9 PUSHI 1
10 STOREG 2
11 PUSHI 1
12 STOREG 1
13 DOCHECK1:
14 PUSHG 1
15 PUSHG 0
16 INFEQ
17 JZ DOEXIT2
18 PUSHG 2
19 PUSHG 1
20 MUL
21 STOREG 2
22 L10:
23 NOP
24 PUSHG 1
25 PUSHI 1
26 ADD
27 STOREG 1
28 JUMP DOCHECK1
29 DOEXIT2:
30 ...
31 STOP
```

10 Testes

A suite de testes em `tests/test_compiler.py` cobre 39 casos, incluindo:

- Os cinco exemplos do enunciado (*Olá Mundo*, *Fatorial*, *Primo*, *Soma de Arrays*, *Conversor de bases*).
- Erros semânticos esperados: variável não declarada, redeclaração, label duplicada, `GOTO` inválido, tipos incompatíveis.
- Detecção e normalização de *fixed-form* e *free-form*, incluindo continuações com `&` e coluna 6.
- Funções e subrotinas com múltiplos argumentos.
- Arrays com acesso e atribuição indexada.

Todos os testes passam (`python -m unittest discover -s tests`).

11 Dificuldades Encontradas

1. **Ambiguidade `ID(expr)`** - sintaticamente, `A(I)` pode ser acesso a array ou chamada de função com um argumento. A gramática trata os dois casos como `ArrayAccess`; a análise semântica e o gerador de código distinguem-nos consultando a tabela de funções.
2. **Detecção de formato** - ficheiros que usam indentação de seis espaços em *free-form* são facilmente confundidos com *fixed-form*. O sistema de pontuação heurística teve de ser calibrado para dar peso decisivo ao marcador `&`.
3. **Inlining com múltiplas chamadas** - quando a mesma função é chamada mais do que uma vez, os rótulos internos gerados têm de ser únicos. Resolvemos com um contador global de rótulos por instância do gerador.
4. **Ciclos `DO` aninhados** - o gerador mantém uma pilha de ciclos activos indexada pela label Fortran; ao encontrar um `LABEL n CONTINUE`, o ciclo correcto é fechado independentemente da profundidade de aninhamento.
5. **Conversão implícita `INTEGER/REAL`** - o Fortran 77 permite misturar inteiros e reais em expressões. A análise semântica aceita estas conversões implícitas e o gerador emite as instruções de conversão correctas (`ITOF/FTOI`).
6. **Ciclos `DO` com variável `REAL`** - o gerador precisa de distinguir o tipo da variável de controlo para escolher os opcodes corretos na condição e no incremento. A solução passou por propagar o tipo da variável até ao contexto do loop (`_ActiveDoLoop`), emitindo `FINFEQ/FADD` para `REAL` e `INFEQ/ADD` para `INTEGER`.

12 Como Correr o Compilador

12.1 Dependências

Listing 4: Instalação de dependências.

```
1 pip install -r requirements.txt # instala ply
```

A única dependência externa é o **PLY** (*Python Lex-Yacc*).

12.2 Compilar um programa Fortran

Listing 5: Uso da linha de comandos.

```
1 python -m src.main <ficheiro.f77> -o <saida.vm>
2
3 # Exemplos:
4 python -m src.main examples/hello.f77      -o hello.vm
5 python -m src.main examples/fatorial_do.f77 -o fatorial.vm
6 python -m src.main examples/primo.f77      -o primo.vm
7 python -m src.main examples/soma_array.f77  -o soma.vm
8 python -m src.main examples/conversor.f77   -o conversor.vm
```

O ficheiro `.vm` gerado pode ser executado diretamente na máquina virtual da unidade curricular.

12.3 Correr os testes

Listing 6: Execução da suite de testes.

```
1 python -m unittest discover -s tests
```

13 Conclusão

O compilador desenvolvido suporta o subconjunto essencial do Fortran 77 especificado no enunciado - tipos base, arrays, controlo de fluxo, entrada/saída, e subprogramas (valorização). A arquitectura modular em seis fases independentes permitiu desenvolver e testar cada componente de forma isolada, resultando numa base de código legível e extensível.

Os 39 testes automatizados cobrem os cinco exemplos padrão do enunciado e os principais casos de erro, garantindo a correcção do compilador dentro do subconjunto implementado.